

Maestría en Ciencias de Datos e Innovación Empresarial

Aprendizaje Automático

(Deep Learning)

Prof. Dr. Marcelo Raponi

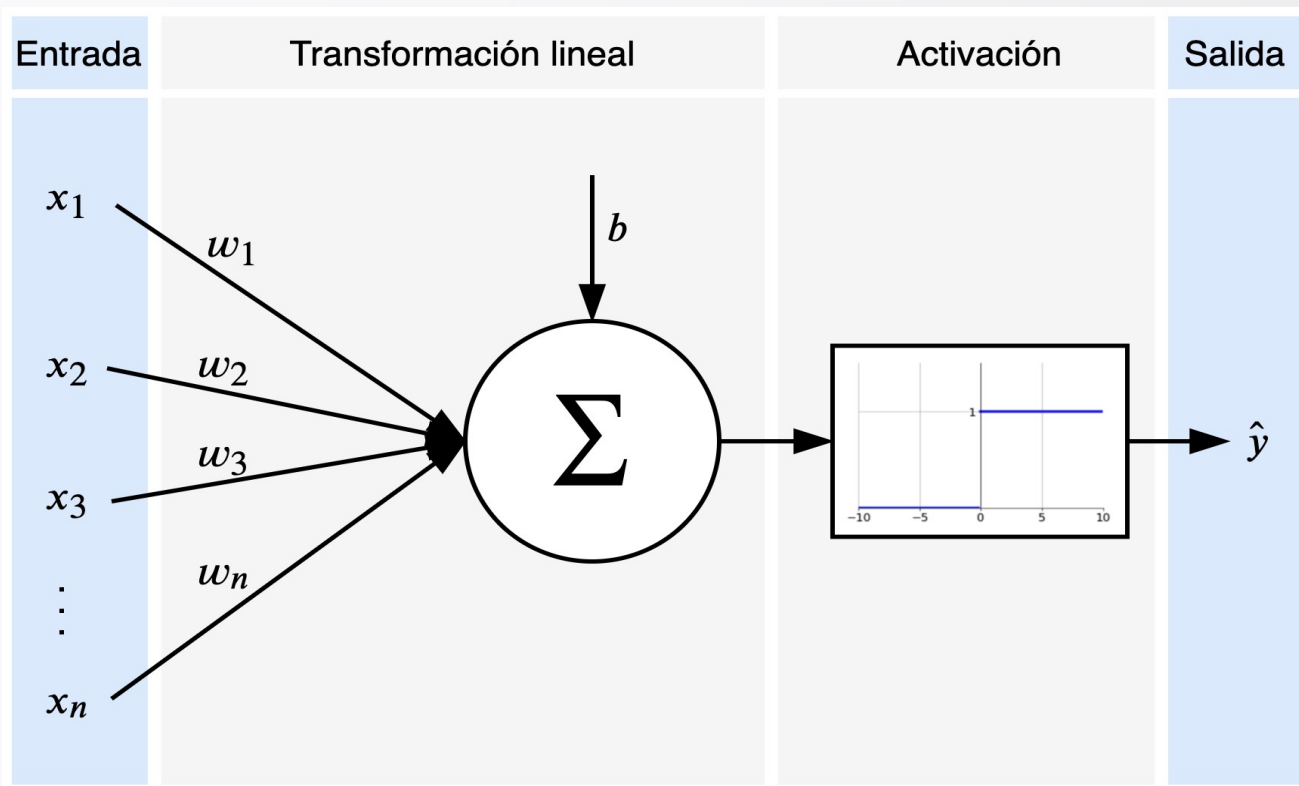
Unidad I: Ejercicios

TEMARIO:

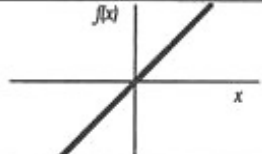
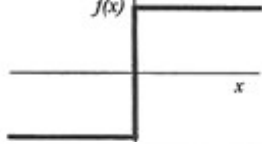
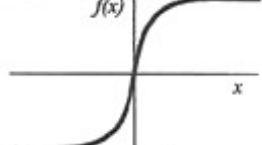
- Regresión Lineal y Regresión Logística.
- Funciones de pérdida clásicas:
 - * Error cuadrático medio (Mean Squared Error, **MSE**).
 - * Entropía cruzada (Cross-Entropy, **CE**).
- Optimizador descenso de gradiente (Gradient Descent, **GD**)
- Optimizador descenso de gradiente estocástico (Stochastic Gradient Descent, **SGD**)
- Funciones de activación
- **Momentum**
- SDG+Momentum para entrenar un clasificador multiclase
- Función **Softmax**
- Función de pérdida (Categorical Cross-Entropy, **CCE**).

Regresor lineal, Regresor logístico, Perceptron

- * Un Perceptron es una neurona artificial basada en el concepto de neurona biológica.
- * En el siguiente diagrama se puede observar la estructura de un Perceptron.
- * La **función de activación** empleada por el Perceptron es el **escalón**.



Funciones de activación

	Función	Rango	Gráfica
Identidad	$y = x$	$[-\infty, +\infty]$	
Escalón	$y = \text{sign}(x)$ $y = H(x)$	$\{-1, +1\}$ $\{0, +1\}$	
Sigmoidea	$y = \frac{1}{1 + e^{-x}}$ $y = \text{tgh}(x)$	$[0, +1]$ $[-1, +1]$	

- * El parámetro **bias** (b) puede ser agregado como una entrada más (x_0) siendo su peso sináptico $w_0=1$.
- * Si la **función de activación** es **lineal** (identidad) se obtiene un **regresor lineal**.
- * Si la **función de activación** es una **sigmoide**, se obtiene un **regresor logístico**.
- * El regresor logístico es más fácil de entrenar (como veremos en la práctica).

PARTE A

- Empezaremos entrenando un **regresor lineal**, es decir, buscaremos los parámetros que minimicen la función de costo o pérdida.
- Una regresión lineal se define como:

$$\hat{y} = x_0 w_0 + x_1 w_1 + \dots + x_i w_i + b$$

siendo:

x: features

w y **b**: parámetros que debemos aprender

- Definimos una función de pérdida (**loss**), que en el caso de la regresión lineal suele ser el **error cuadrático medio (MSE)**:

$$L(y, \hat{y}) = MSE(y, \hat{y}) = \frac{1}{N} \sum (y - \hat{y})^2$$

siendo:

y: valores esperados(target)

y_hat: salidas predichas por el regresor

N: cantidad de muestras

- En primer lugar trabajaremos con una **regresión lineal simple (caso univariado)**:

$$\hat{y} = x \cdot w + b$$

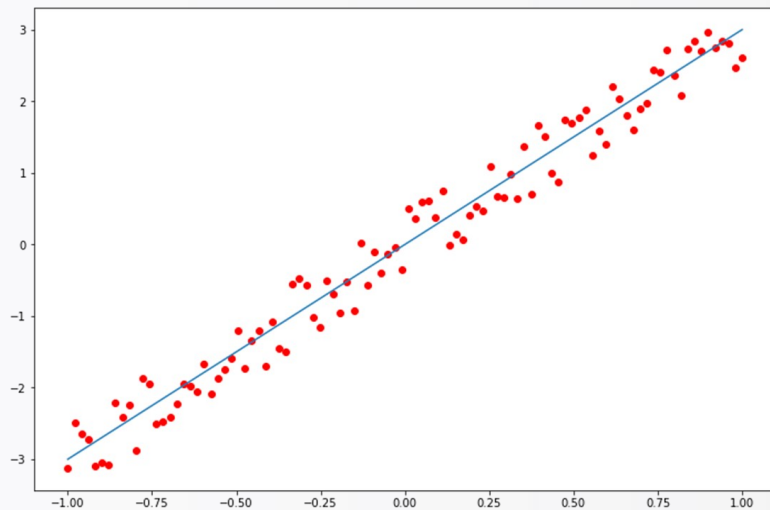
Objetivo: buscar los valores de w y b que minimicen la función de pérdida:

$$\arg \min_{w, b} L(y, xw + b)$$

- Por simplicidad, generaremos 100 datos artificiales aplicando la siguiente ecuación:

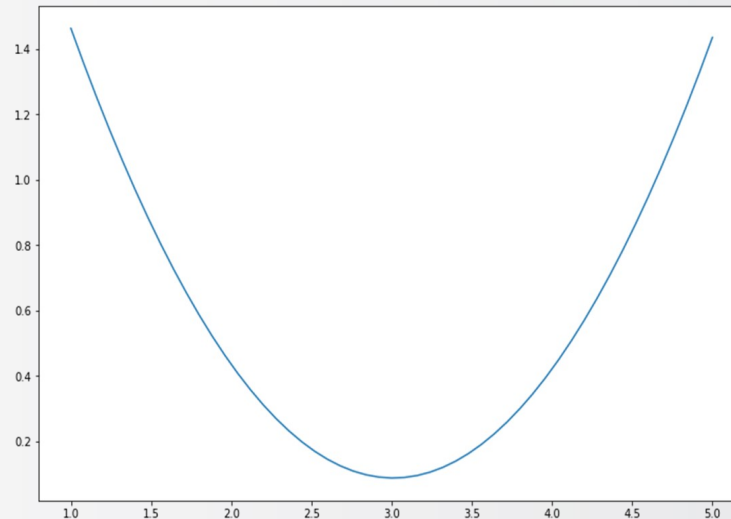
$$y = 3 \cdot x + (\text{rand} - 0.5)$$

- Para buscar los valores de w y b que minimizan el error, podríamos probar manualmente con distintos valores y quedarnos con aquellos que minimicen el error.
- Cada prueba requiere calcular la pérdida sobre todos los elementos en el conjunto de entrenamiento.
- Es imposible evaluar sobre todos los valores de w y b , por lo tanto, necesitamos una mejor estrategia para buscar los mejores valores de dichos parámetros.
- Si aplicamos una regresión lineal con $w = 3$ y $b = 0$ obtenemos una recta que ajusta los datos con un cierto valor de MSE.



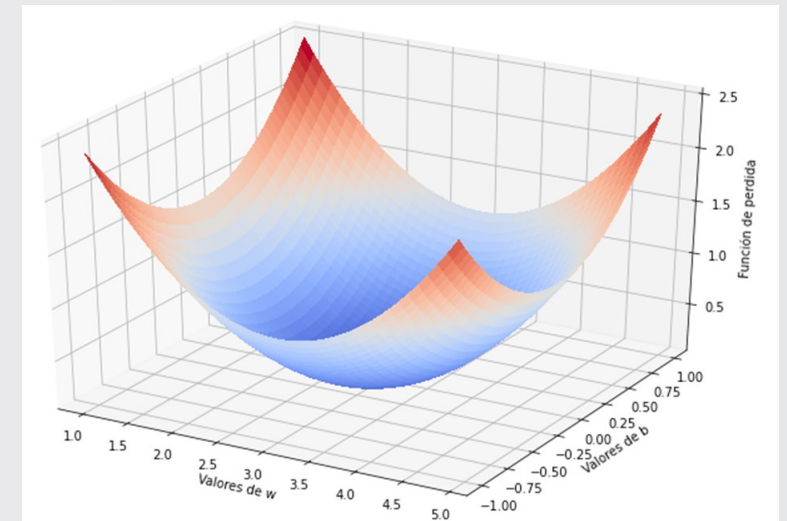
Nube de puntos y recta de ajuste

Ejercicio A-1



Loss en función de w (b=0)

Ejercicio A-2

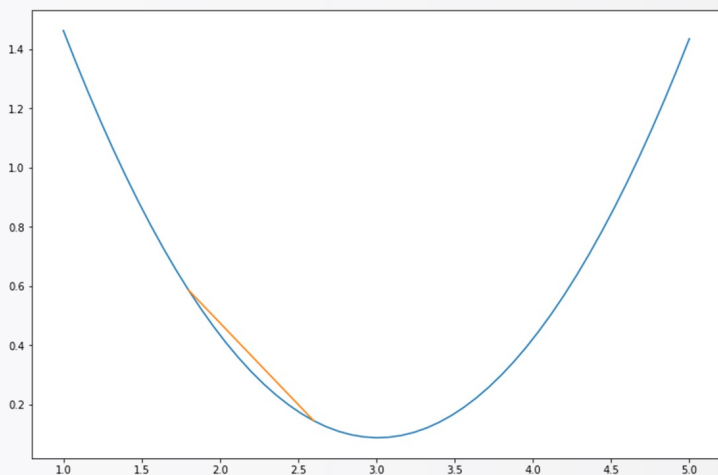


Loss en función de w y b

Ejercicio A-3

- Para simplificar el análisis fijamos $b = 0$.
- Podríamos probar con 2 valores cercanos de w y determinar cómo varia la función de perdida.
- Si trazamos una recta entre estos dos valores, su pendiente puede ser calculada como:

$$pendiente(w, \Delta) = \frac{L(w) - L(w + \Delta)}{\Delta}$$



Observando el gráfico, intuitivamente se puede afirmar que:

- Si la **pendiente es negativa**, hay que **incrementar** el valor de w para disminuir el error.
- Si la **pendiente es positiva**, hay que **decrementar** el valor de w para disminuir el error.

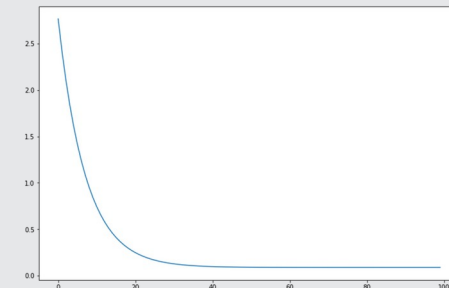
```
b = 0
epoch = 0
w = random()
```

```
while epoch < MAX_EPOCH:
    epoch += 1
    #Calcular pendiente
    pw = pendiente_w(y, x, w, b, delta)
    #Actualizar w
    w = w - lr * pw
```

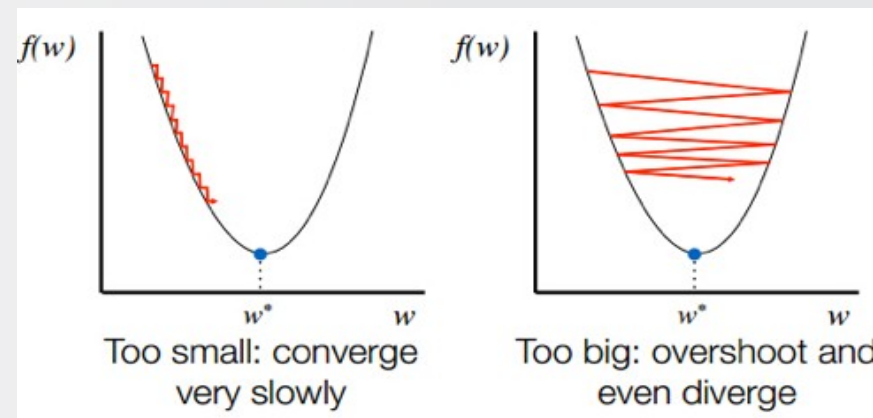
Hiper-parámetros:

- Epochs
- Delta
- Learning rate

Aplicando el algoritmo veremos que el valor del error va disminuyendo a medida que avanza el entrenamiento.



El **step** de la búsqueda del mínimo de la loss depende del hiperparámetro learning rate (**lr**) y del valor del gradiente (**pw**)



- El algoritmo anterior puede generalizarse para todos los parámetros de nuestro modelo (w y b , que son **escalares**).
- Considerando las pendientes, el valor deseado de delta es el menor posible, por lo tanto:

$$\lim_{\Delta \rightarrow 0} \text{pendiente } e_w(w, \Delta) = \lim_{\Delta \rightarrow 0} \frac{L(w) - L(w + \Delta)}{\Delta} = \frac{dL}{dw}$$

$$\lim_{\Delta \rightarrow 0} \text{pendiente } e_b(b, \Delta) = \lim_{\Delta \rightarrow 0} \frac{L(b) - L(b + \Delta)}{\Delta} = \frac{dL}{db}$$

- Calcularemos estas derivadas, además de implementar nuestro modelo y la función de pérdida, utilizando el framework **TensorFlow**.

```
epoch = 0
w = random()
b = random()

while epoch < MAX_EPOCH:
    epoch += 1

    #Calcular pendiente
    pw = pendiente_w(y, x, w, b, delta)
    pb = pendiente_b(y, x, w, b, delta)

    #Actualizar w y b
    w = w - lr * pw
    b = b - lr * pb
```

```
import tensorflow as tf
```



Importa la librería de TF

```
def lineal(x, w, b):
    return x @ w + b
```



Calcula la salida del regresor, haciendo el producto escalar

```
def mse(y_true, y_pred):
    return tf.math.reduce_mean((y_true - y_pred) ** 2)
```



Determina el MSE

- **TensorFlow**: librería de matemática orientada al ML desarrollada por Google y utilizada por diferentes empresas.
- Fácilmente utilizable desde Google Colab. Soporta procesamiento en TPUs y GPUs para acelerar la computación.
- Integra **Keras**, una librería para implementar redes neuronales de forma simplificada.

```
#Tipos de datos esperados por tf
```

```
x = x.astype(np.float32)
```

```
y = y.astype(np.float32)
```

TensorFlow asume que el tipo de dato es np.float32

```
w = tf.random.uniform(shape=[], minval=-1, maxval=1)
```

```
b = tf.random.uniform(shape=[], minval=-1, maxval=1)
```

Inicializa los parámetros como arreglos tf

```
epochs = 100
```

```
lr = 0.1
```

```
Losses = []
```

```
for i in range(epochs):
```

```
    with tf.GradientTape() as g:
```

```
        g.watch([w, b])
```

```
        loss = mse(y, lineal(x, w, b))
```

```
        losses.append(loss.numpy())
```

```
    gw, gb = g.gradient(loss, [w, b])
```

```
    w = w - lr * gw
```

```
    b = b - lr * gb
```

Graba los gradientes de las variables w y b

tensor.numpy() extrae el valor

Calcula los gradientes

Actualiza los parámetros

PARTE B Caso de aplicación: Breast Cancer dataset de la Universidad de Wisconsin.

- Realizaremos un split del dataset: 500 training y 69 testing.
- Reemplazamos la **función de activación** identidad del regresor lineal por una **sigmoide**:
- Obtenemos un **regresor logístico** que usaremos para **clasificación binaria** (maligno, benigno).

$$\text{sigmoid}(z) = \frac{1}{1 + e^{-z}}$$

```
def logisticRegresion(x, w, b):  
    return 1/(1+tf.exp(-(tf.matmul(x, w) + b)))[0]
```

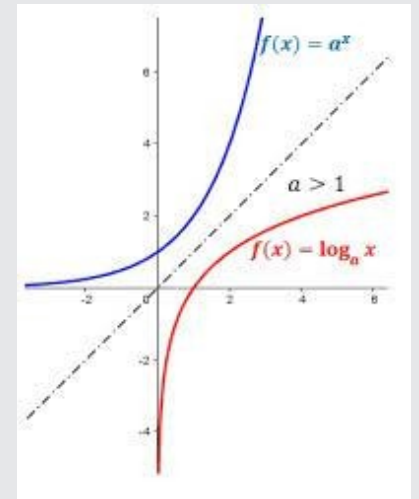
- La función de pérdida a emplear es la **binary cross-entropy**:

$$L(y, \hat{y}) = BCE(y, \hat{y}) = \frac{\sum (-y \cdot \log(\hat{y}) - (1 - y) \cdot \log(1 - \hat{y}))}{n}$$

```
def binary_crossentropy(yt, yp):  
    return tf.math.reduce_mean(-yt*tf.math.log(yp) - (1-yt)*tf.math.log(1-yp))
```

- Podríamos aplicar el mismo algoritmo que vimos para la regresión lineal:

```
w = tf.random.uniform(shape=[30, 1], minval=-1, maxval=1)  
b = tf.random.uniform(shape=[], minval=-1, maxval=1)  
epochs = 100  
lr = 0.1  
losses = []  
for i in range(epochs):  
    with tf.GradientTape() as g:  
        g.watch([w, b])  
        y_hat = logisticRegresion(x_train, w, b)  
        loss = binary_crossentropy(y_train, y_hat)  
        losses.append(loss.numpy())  
    gw, gb = g.gradient(loss, [w, b])  
    w = w - lr * gw  
    b = b - lr * gb
```



Propiedad	Valor
Clases	2
Ejemplos por clase	212(M-0), 357(B-1)
Total de instancias	569
Dimensionalidad	30

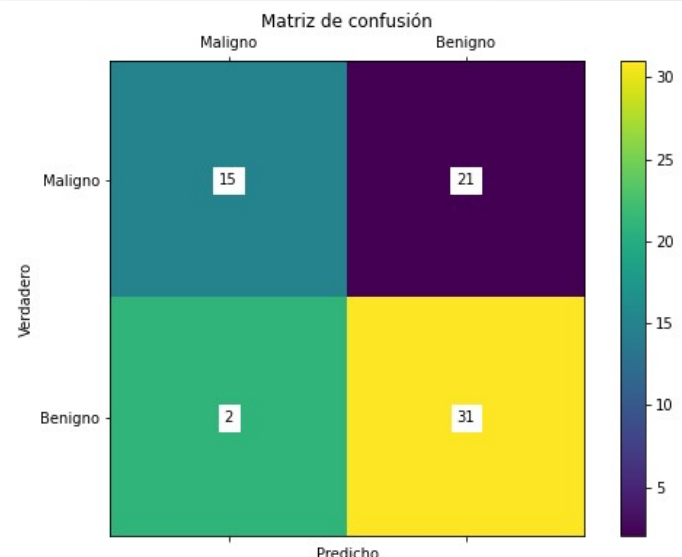
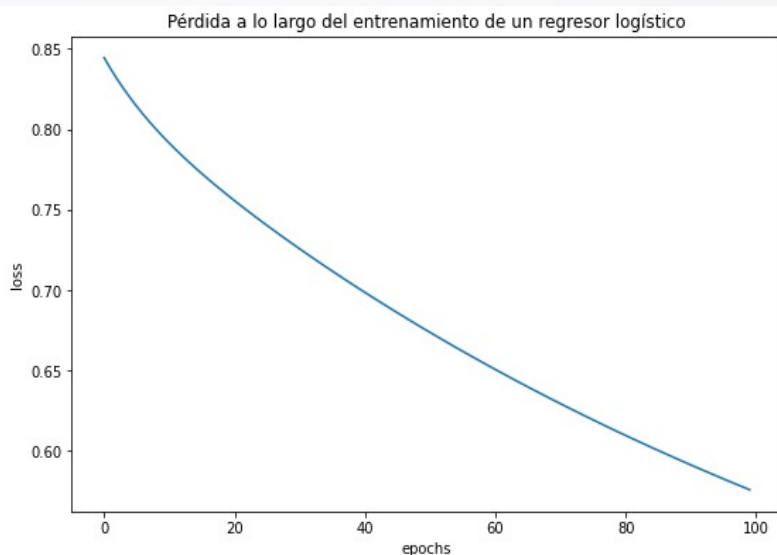
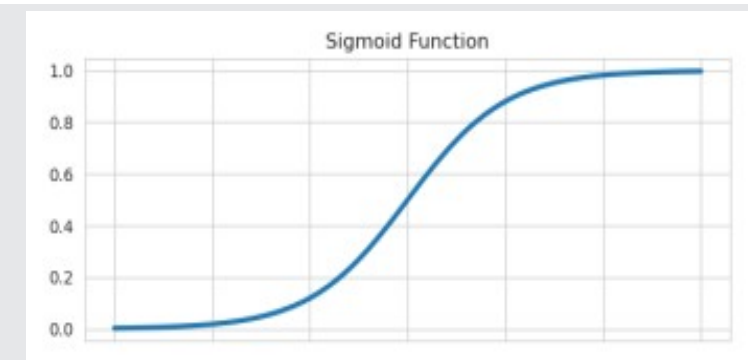
Atención: probablemente algunos valores de w y b se volverán **NaN**. ¿Por qué?

- Los features pueden tomar valores muy variados (grandes y pequeños)
- Si bien la **sigmoide es asintótica al 0 y 1**, cuando se utiliza una representación de punto flotante, el valor se aproxima. Valores muy pequeños se aproximarán al valor 0.
- Entonces, al tener que calcular el **log(0)** nos generaría **NaN**.

Solución:

- Acotar los valores de los features escalando los mismos al intervalo $[0, 1]$ (normalizar, estandarizar). Esta solución facilita el aprendizaje ya que utilizamos un solo learning rate para todos los parámetros.
- **Clippear** (recortar) los valores de la salida predicha.

```
def binary_crossentropy(yt, yp):  
    return tf.math.reduce_mean(-yt*tf.math.log(tf.clip_by_value(yp, 1e-6, 1)) - (1-yt) *  
                                tf.math.log(tf.clip_by_value(1-yp, 1e-6, 1)))
```



Stochastic Gradient Descent :

- Aproxima el gradiente de todo el dataset actualizando sobre muestras aleatorias del mismo.
- Reduce la carga computacional ya que no requiere realizar operaciones sobre todo el dataset.
- En general es más rápido, ya que actualiza los parámetros más frecuentemente.
- No en todos los pasos genera una mejor solución.

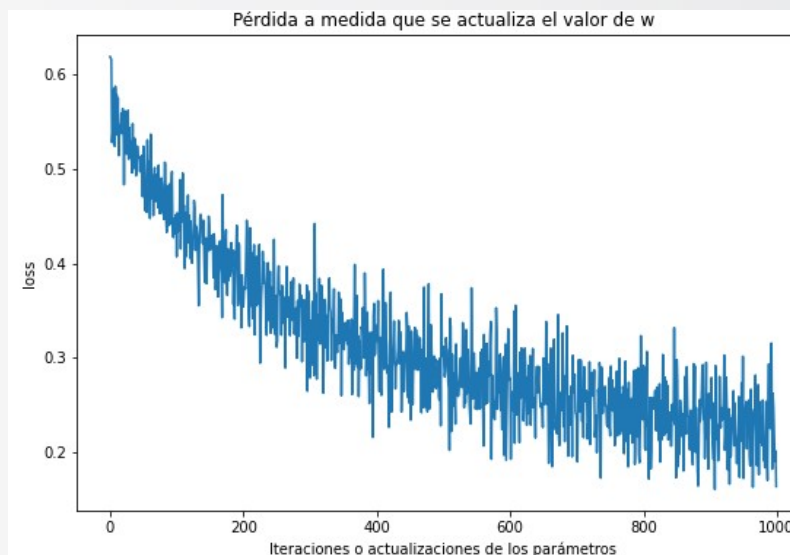
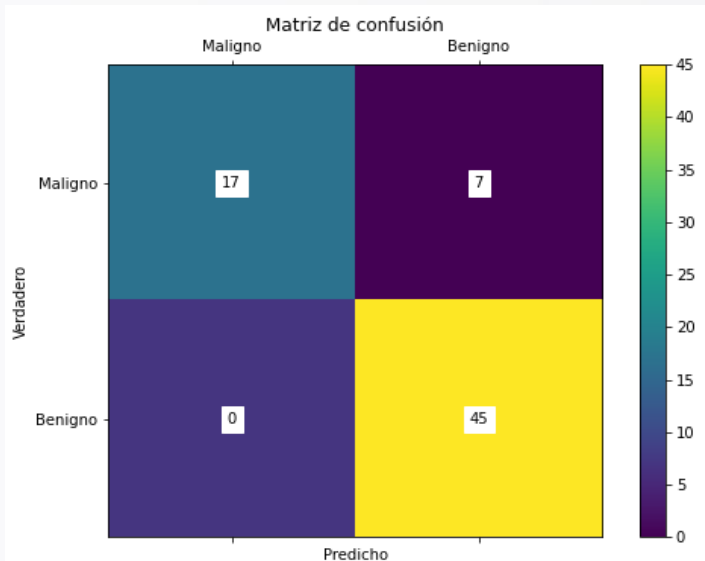
Stochastic Gradient Descent:

```
w = tf.random.uniform(shape=[30, 1], minval=-1, maxval=1)
b = tf.random.uniform(shape=[], minval=-1, maxval=1)
epochs = 100
lr = 0.1
losses = []
```

```
for i in range(epochs):
    x_s, y_s = shuffle(x_train, y_train)
    for mb in range(0, 500, 50):
        with tf.GradientTape() as g:
            g.watch([w, b])
            y_hat = logisticRegresion(x_s[mb:mb+50], w, b)
            loss = binary_crossentropy(y_s[mb:mb+50], y_hat)
            losses.append(loss.numpy())
        gw, gb = g.gradient(loss, [w, b])
        w = w - lr * gw
        b = b - lr * gb
```

Desordena las
instancias para el
entrenamiento

Actualiza los pesos
por cada mini batch



- La pérdida para cada mini-batch varía notablemente cuando se lo compara con sus vecinos (aspecto ruidoso de la curva azul).
- Hay una clara tendencia a que la pérdida disminuya en el tiempo.
- Con esta técnica se logra mejorar los resultados del entrenamiento.

Momentum: es una técnica que permite mejorar el entrenamiento y se basa en que:

- No todos los mini-batches aportan al entrenamiento.
- En general, hay una tendencia a disminuir el error.
- Si se considera el histórico, se tiene mejor información de **cuánto** y **en qué dirección** se deben actualizar los parámetros.
- Actualizaciones más recientes deben tener más importancia y aquellas que ocurrieron hace mucho, no deberían tener un peso significativo en nuevas actualizaciones.
- Las actualizaciones se hacen indirectamente a través de **velocidades**, las cuales son actualizadas con cada mini-batch.

```
vw = tf.zeros(shape=[30, 1])  
vb = tf.zeros(shape=[])
```

Velocidades iniciales son cero

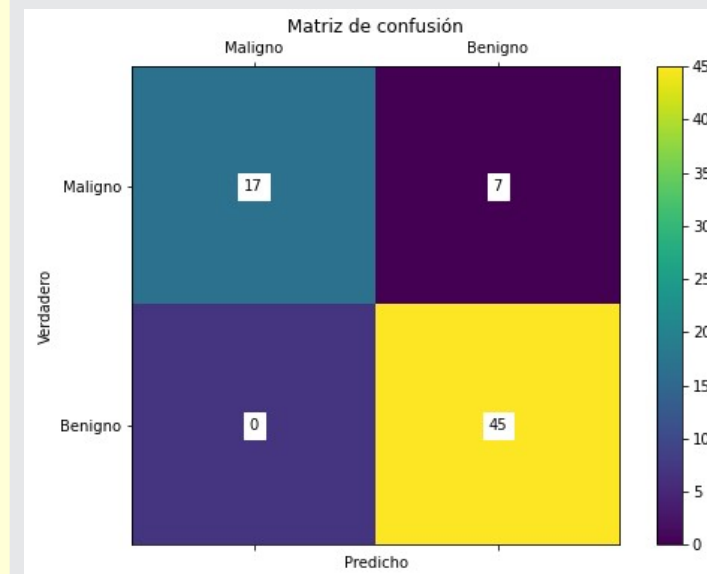
```
momentum = 0.9
```

Momentum: peso del factor histórico

```
for i in range(epochs):  
    x_s, y_s = shuffle(x_train, y_train)  
    for mb in range(0, 500, 50):  
        with tf.GradientTape() as g:  
            g.watch([w, b])  
            y_hat = logisticRegresion(x_s[mb:mb+50], w, b)  
            loss = binary_crossentropy(y_s[mb:mb+50], y_hat)  
            losses.append(loss.numpy())  
            gw, gb = g.gradient(loss, [w, b])  
            vw = momentum * vw - lr * gw  
            vb = momentum * vb - lr * gb  
            w = w + vw  
            b = b + vb
```

Actualización de la velocidad
basada en los gradientes

Actualización de los pesos basada
en la velocidad actualizada

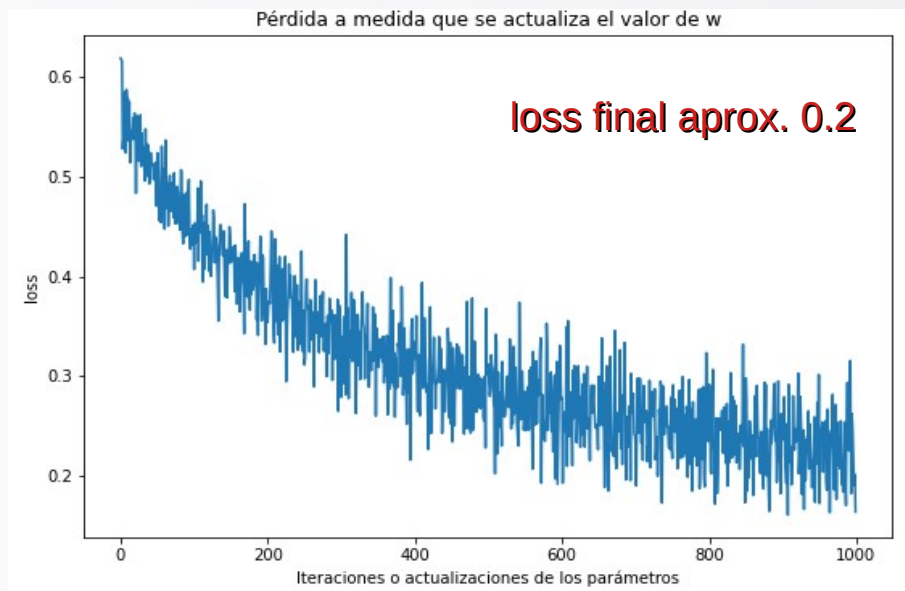


- Con esta técnica vuelven a aparecer los **NaN**, ya que como se mejoran los valores, las predicciones se acercan a 0 o 1.
- Para evitar este problema se suelen recortar los valores extremos (clipping).

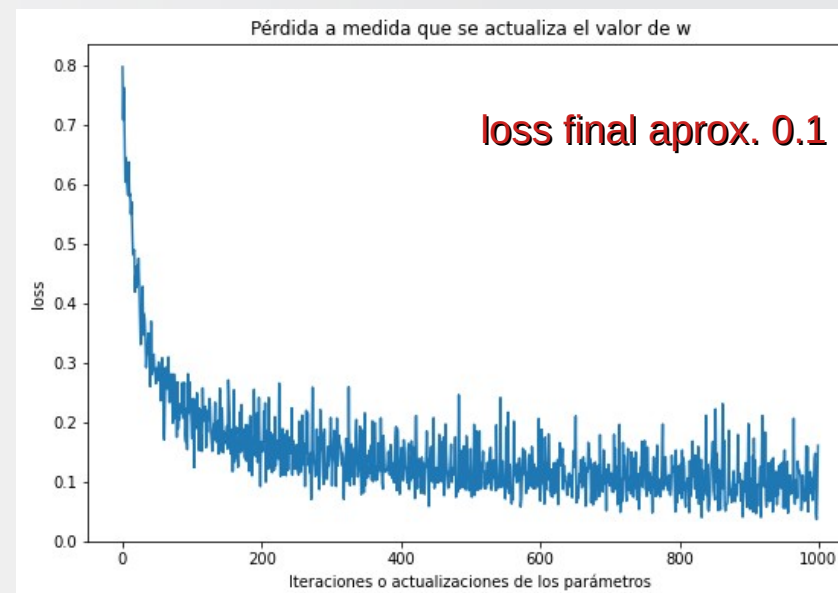
```
def crossentropy(yt, yp):  
    return tf.math.reduce_mean(-yt*tf.math.log(tf.clip_by_value(yp, 1e-6, 1)) -  
                               (1-yt)*tf.math.log(tf.clip_by_value(1-yp, 1e-6, 1)))
```

yp min = 1e-6
yp max = 1

Entrenamiento sin momentum:



Entrenamiento con momentum:



Ventajas del momentum:

- El entrenamiento es más rápido.
- Se llegan a menores valores de pérdida.
- Mejora la convergencia.
- No agrega complejidad computacional.

Funciones de activación:

Tanh

ReLU

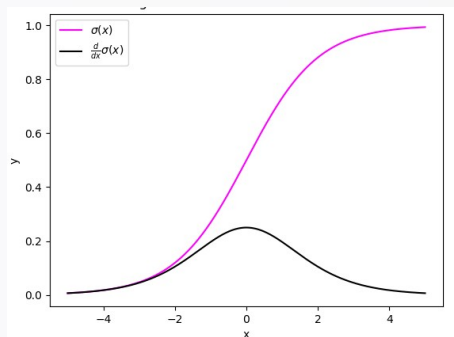
Leaky ReLU

ELU

Softmax

SoftPlus, etc.

<https://ml-explained.com/blog/activation-functions-explained>



Clasificación multiclase

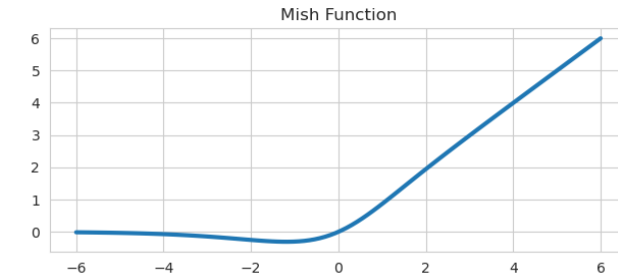
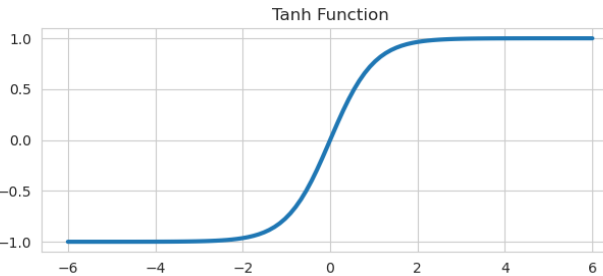
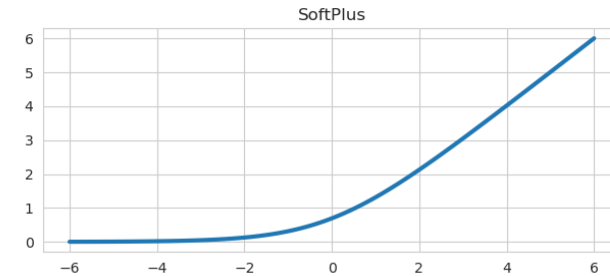
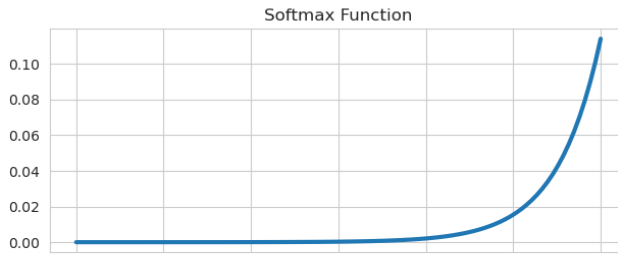
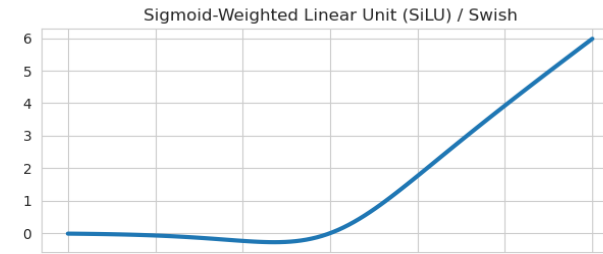
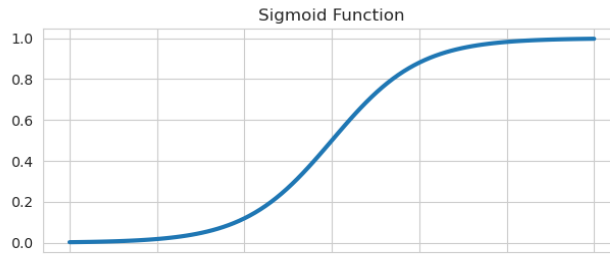
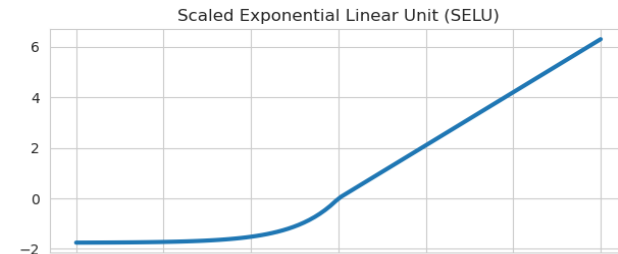
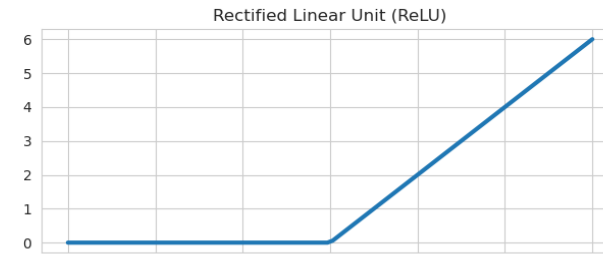
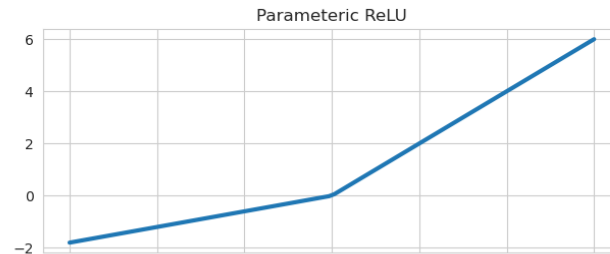
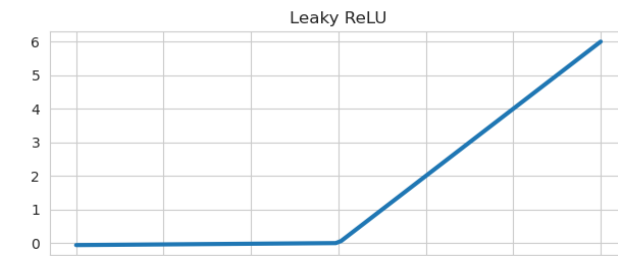
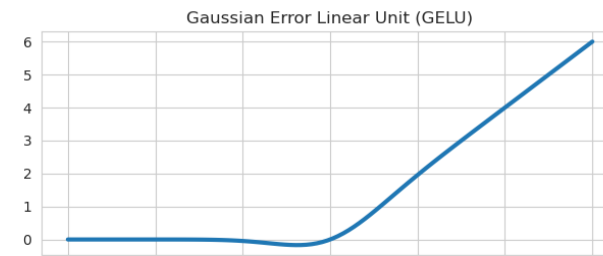
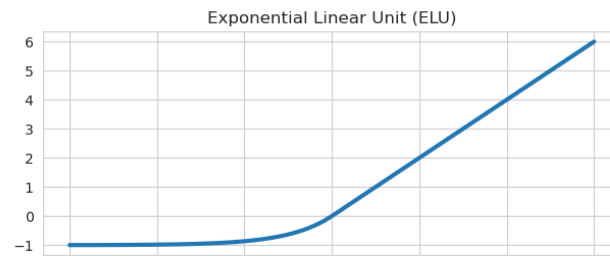
Softmax: función que se aplica de forma vectorial

- Usada para **clasificación multi-clase**.
- Asigna un **valor entre 0 y 1** a c/elemento del vector.
- La **suma** total de los elementos es **1**.
- **Asigna una probabilidad a cada clase.**

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}$$

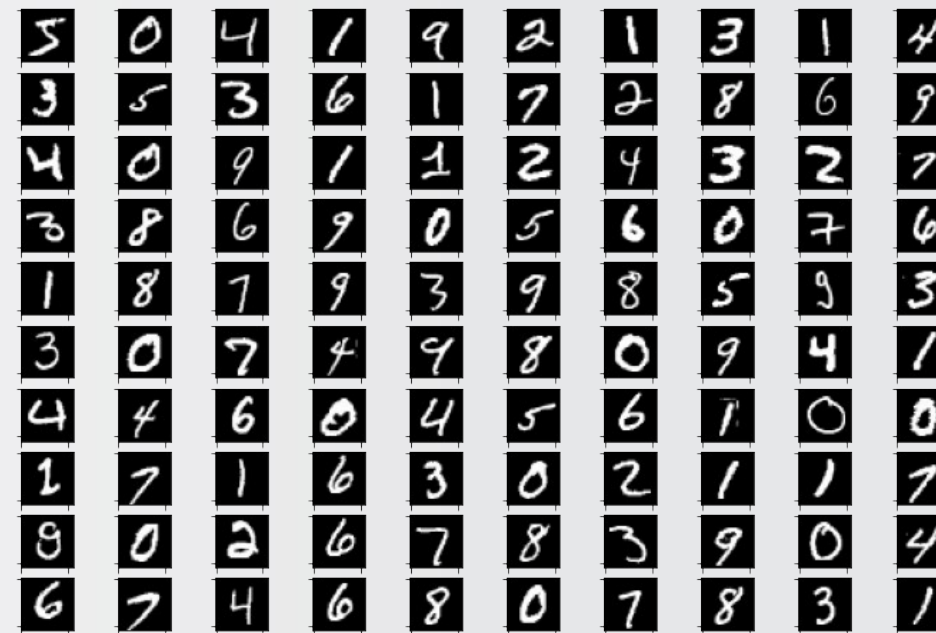
$$\mathbf{z} = \begin{pmatrix} z_1 \\ z_2 \\ z_3 \\ z_4 \end{pmatrix} \left\{ \begin{array}{l} 1.1 \rightarrow \\ 2.2 \rightarrow \\ 0.2 \rightarrow \\ -1.7 \rightarrow \end{array} \right. \left. \begin{array}{l} \rightarrow 0.224 \\ \rightarrow 0.672 \\ \rightarrow 0.091 \\ \rightarrow 0.013 \end{array} \right. \mathbf{s} = \begin{pmatrix} s_1 \\ s_2 \\ s_3 \\ s_4 \end{pmatrix}$$

Diagram illustrating the Softmax function applied to a vector $\mathbf{z}$. The inputs are $z_1=1.1, z_2=2.2, z_3=0.2, z_4=-1.7$. These are processed by the formula $s_i = \frac{e^{z_i}}{\sum_l e^{z_l}}$ to produce the output vector $\mathbf{s}$ with values $s_1=0.224, s_2=0.672, s_3=0.091, s_4=0.013$.



Clasificación Multiclase: Dataset MNIST

Propiedad	Valor
Clases	10
Tamaño de las imagenes	28 X 28 (784)
Instancias de entrenamiento	60.000
Instancias de entrenamiento	10.000
Valor mínimo de cada pixel	0
Valor máximo de cada pixel	255



Las instancias están representadas por matrices de dimensión **28x28**.

Podemos mapear las matrices en vectores.

Cada pixel tiene un valor entre 0 y 255.

Dividimos por 255 para llevar los valores al rango [0-1]

Las clases están representadas por un número entre 0 y 9.

A la salida de una softmax debemos cambiar la representación a **one-hot encoding**.

- 2 → (0,0,1,0,0,0,0,0,0,0)
- 8 → (0,0,0,0,0,0,0,0,1,0)

```
size = x_train.shape[1]*x_train.shape[2] → 28*28=784
```

```
x_train = x_train.reshape((x_train.shape[0], size))/255
```

```
x_test = x_test.reshape((x_test.shape[0], size))/255
```

```
yc_train = to_categorical(y_train)
```

```
yc_test = to_categorical(y_test)
```

→ **one-hot encoding**

Ejercicio B-2

```
x_train = x_train.astype(np.float32)
```

```
x_test = x_test.astype(np.float32)
```

```
yc_train = yc_train.astype(np.float32)
```

```
yc_test = yc_test.astype(np.float32)
```

Categorical Cross-Entropy:

- Para clasificación, puede utilizarse la función de pérdida cross-entropy, pero la versión vista hasta ahora era binaria.
- La función de pérdida que emplearemos en la tarea de **clasificación multiclase** es conocida con el nombre de **categorical cross-entropy** (CCE) y se define como:

$$CCE = L(y, \hat{y}) = \frac{-\sum y \log \hat{y}}{n} = \frac{-(0, \dots, 1, \dots, 0) \log(\hat{y}_1) - \dots - (0, \dots, 1, \dots, 0) \log(\hat{y}_n)}{n}$$



$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}$$

```
def softmax(z):  
    exp = tf.exp(z)  
    return exp / tf.expand_dims(tf.math.reduce_sum(exp, axis=1), axis=-1)  
  
def predict(x, w, b):  
    return softmax(tf.matmul(x, w) + b)  
  
def categorical_crossentropy(y_true, y_pred):  
    return tf.math.reduce_mean(-tf.math.reduce_sum(y_true *  
                                                    tf.math.log(tf.clip_by_value(y_pred, 1e-6, 1)), axis=1))
```

```
w = tf.random.uniform(shape=[size, 10], minval=-1, maxval=1)
b = tf.random.uniform(shape=[10], minval=-1, maxval=1)
```

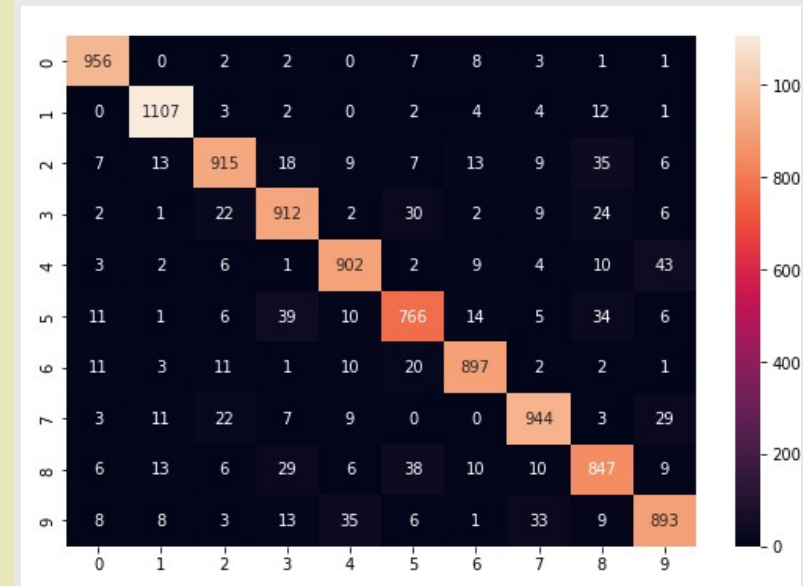
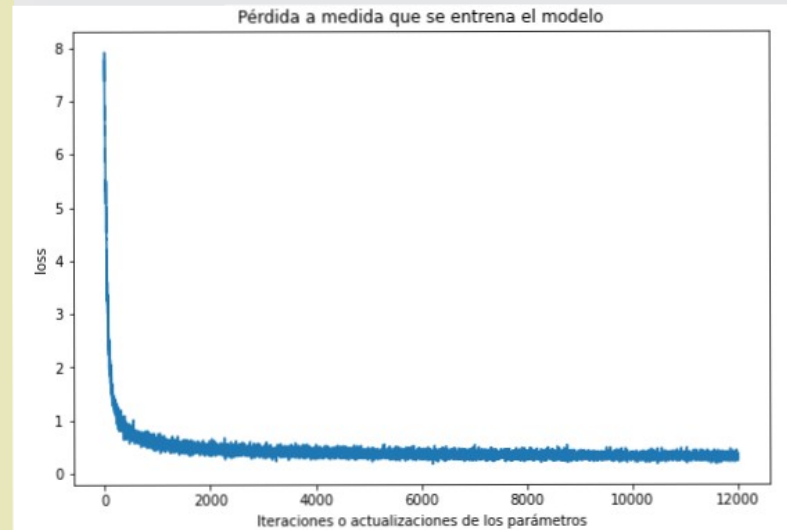
```
# hiperparámetros
epochs = 100
lr = 0.01
momentum = 0.9
batch_size = 500
```

```
losses = []
vw = tf.zeros(shape=[size, 10])
vb = tf.zeros(shape=[10])
```

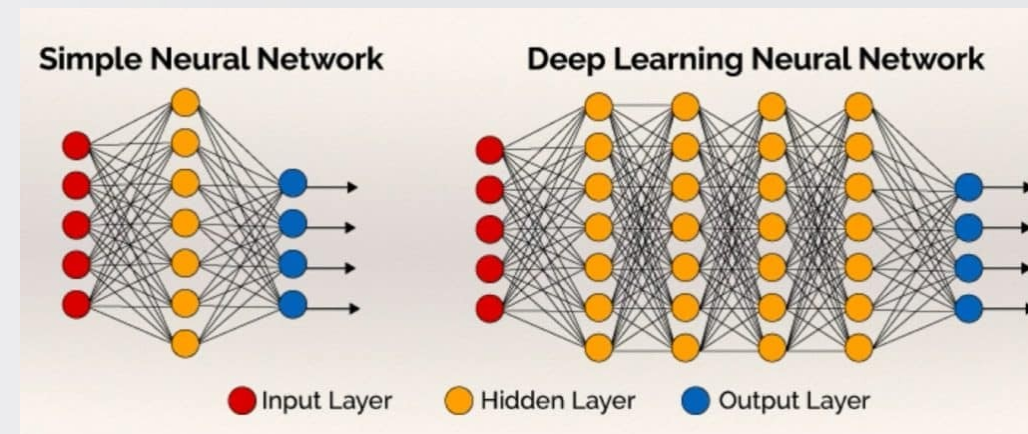
```
for i in tqdm(range(epochs)):
    x_s, y_s = shuffle(x_train, yc_train)
    for mb in range(0, x_s.shape[0], batch_size):
        with tf.GradientTape() as g:
            g.watch([w, b])
            y_pred = predict(x_s[mb:mb + batch_size], w, b)
            y_true = y_s[mb:mn + batch_size]

            loss = categorical_crossentropy(y_true, y_pred)
            losses.append(loss.numpy())
```

```
gw, gb = g.gradient(loss, [w, b])
vw = momentum * vw - lr * gw
vb = momentum * vb - lr * gb
w = w + vw
b = b + vb
```



- Trabajaremos con el **dataset MNIST** y compararemos los resultados obtenidos por el clasificador multiclase de la parte B.
- Usaremos el framework TensorFlow-Keras en primer lugar para construir nuestra red neuronal.
- La red tendrá **784 entradas** (features), **10 bias** y **10 neuronas** formando una capa.
- Cada feature se conecta con todas las neuronas (full-connected, o capa **densa**)
- En total habrá **7.850 parámetros** para aprender.



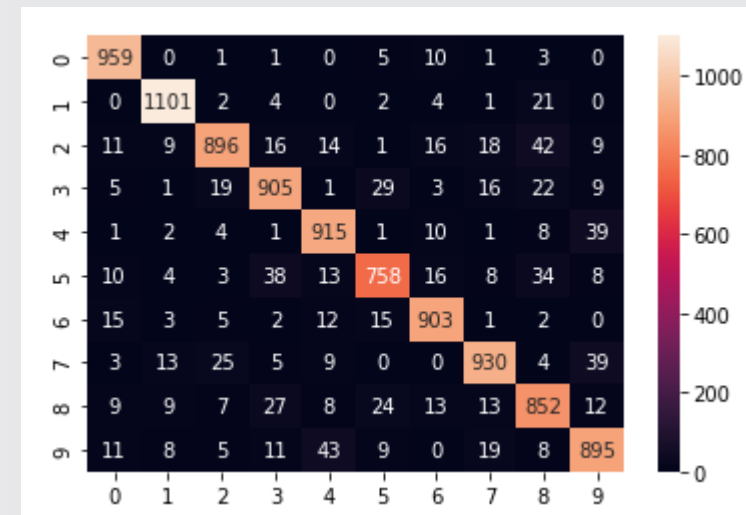
```
BS = 500 #batch-size
```

```
i = Input((size,))
d = Dense(10, activation='softmax')(i)
```

```
model = Model(inputs=i, outputs=d)
model.compile(loss='categorical_crossentropy', optimizer='sgd')
model.summary()
```

```
model.fit(x_train, yc_train, batch_size=BS, epochs=100)
y_pred = model.predict(x_test)
```

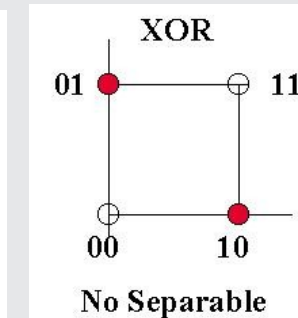
```
y_pred = np.argmax(y_pred, axis=1)
cm = confusion_matrix(y_test, y_pred)
```



Problema del XOR

- La función booleana XOR (or exclusivo) no puede ser aprendida por una regresión logística.
- Generamos un **regresor logístico** usando una RNA de una neurona, con función de activación **sigmoide**, función de pérdida **binary cross-entropy**, y como optimizador emplearemos **nadam**.
- Entrenaremos por **1000 epochs**. Como se puede observar en los gráficos, una sólo perceptron no puede resolver la tarea.

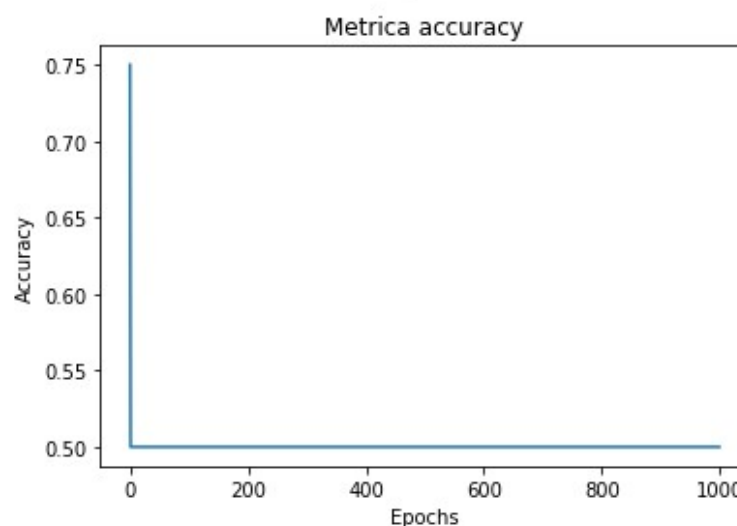
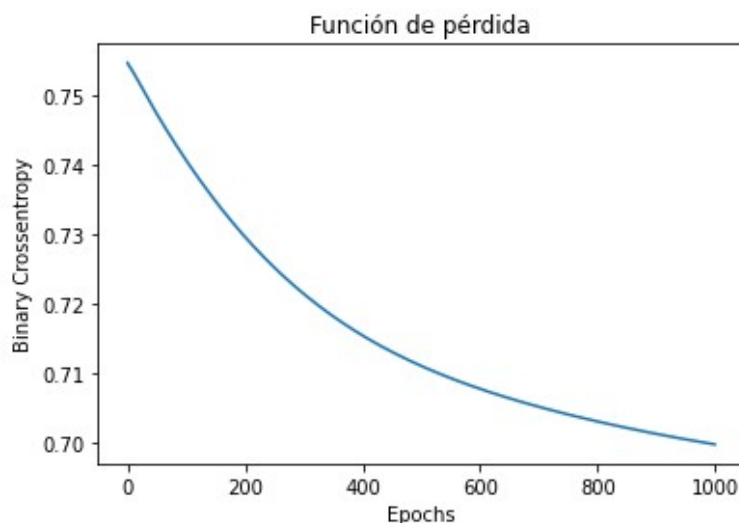
X_0	X_1	Y
0	0	0
0	1	1
1	0	1
1	1	0



```
i = Input((2,)) #2 entradas, son las coordenadas de los puntos
d = Dense(1, activation='sigmoid')(i)

model = Model(i, d)
model.compile(loss='binary_crossentropy', optimizer='nadam', metrics=['accuracy'])

h = model.fit(x, y, epochs=1000, verbose=0)
```



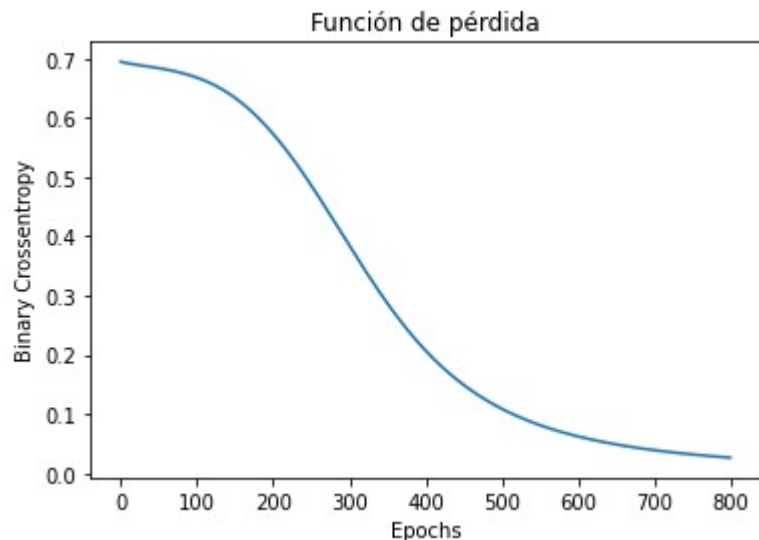
- Modifiquemos nuestra red neuronal para ver si resuelve el problema del XOR.
- Ahora nuestra capa oculta tiene **30 neuronas** con activación **tanh**
- La capa de salida tiene **1 neurona** con activación **sigmoide**
- Usaremos cómo función de pérdida la **binary cross_entropy** y el mismo optimizador (**nadam**).

```
i = Input((2,)) #entradas
d = Dense(30, activation='tanh')(i) #capa de 30 neuronas, función de activación tangente hiperbólica
net = Dense(1, activation='sigmoid')(d) #capa de 1 neurona, función de activación sigmoide.

model = Model(i, net)

model.compile(loss='binary_crossentropy', optimizer='nadam')

h = model.fit(x, y, epochs=800, verbose=0)
```



```
0 xor 0 -> 0.023104913532733917
0 xor 1 -> 0.974748969078064
1 xor 0 -> 0.9702503681182861
1 xor 1 -> 0.02612299472093582
```

Cómo se puede observar en el gráfico, la función de pérdida disminuye con cada epoch, y al final, los valores obtenidos de aplicar la red neuronal a nuestros datos demuestran que logran resolver el problema XOR

A continuación, aplicaremos una red MLP al dataset MNIST para realizar la tarea de clasificación multi-clase.

- Modifiquemos nuevamente nuestra red neuronal para poder resolver la tarea de **clasificación multiclase**, con el dataset **MNIST**.
- Ahora nuestra capa oculta tiene **200 neuronas** con activación **lineal**
- La capa de salida tiene **10 neuronas** con activación **softmax**
- Usaremos como función de pérdida la **categorical_crossentropy** y el optimizador (**SGD**).
- Total de parámetros entrenables: **159.010**

```
i = Input((x_train.shape[1],)) #784 entradas
d = Dense(200)(i)
d = Dense(10, activation='softmax')(d)

model = Model(i, d)

model.compile(loss='categorical_crossentropy', optimizer='sgd')

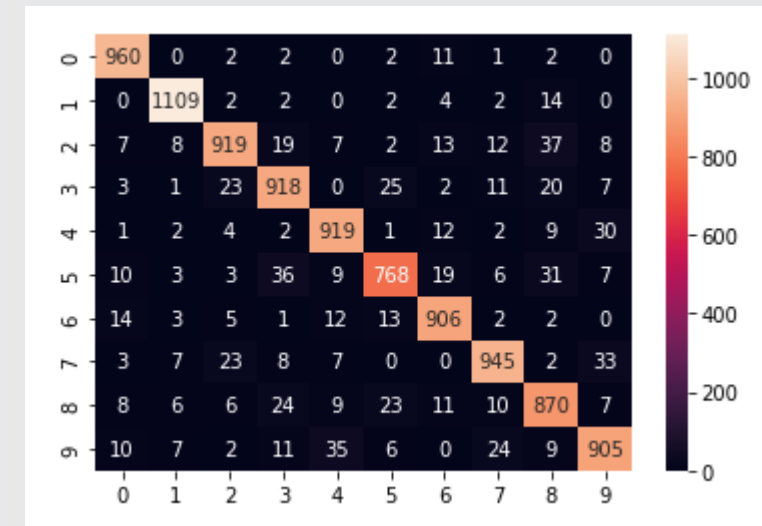
model.summary()

model.fit(x_train, yc_train, batch_size=BS, epochs=100)
#yc_train (target codificados)

y_pred = model.predict(x_test)

y_pred = np.argmax(y_pred, axis=1)

cm = confusion_matrix(y_test, y_pred)
```



	precision	recall	f1-score	support
0	0.94	0.98	0.96	980
1	0.97	0.98	0.97	1135
2	0.93	0.89	0.91	1032
3	0.90	0.91	0.90	1010
4	0.92	0.94	0.93	982
5	0.91	0.86	0.89	892
6	0.93	0.95	0.94	958
7	0.93	0.92	0.93	1028
8	0.87	0.89	0.88	974
9	0.91	0.90	0.90	1009
accuracy			0.92	10000
macro avg	0.92	0.92	0.92	10000
weighted avg	0.92	0.92	0.92	10000

- Utilicemos ahora el optimizador SGD, seteando dos parámetros adicionales: **momentum** y **Nesterov**
- Además, usaremos como función de pérdida la **sparse_categorical_crossentropy**, la cual se emplea cuando los target son numéricos, es decir, no están codificado one-hot

```
model.compile(loss='sparse_categorical_crossentropy', optimizer=(SGD(momentum=0.9, nesterov=True)))

y_pred = model.predict(x_test)

y_pred = np.argmax(y_pred, axis=1)

cm = confusion_matrix(y_test, y_pred)
```

Framework PyTorch

- En el siguiente ejemplo veremos como implementar la misma red utilizando otro framework diferente. En lugar de TensorFlow/Keras usaremos PyTorch.
- Si bien tiene similitudes, también tiene muchas diferencias.
- Primeramente importamos torch y preguntamos si tenemos una GPU (Graphics Processing Unit) disponible y el toolkit de CUDA (Compute Unified Device Architecture) instalado, lo cual nos permitirá hacer cómputo paralelo.

```
import torch
from torch import nn

if torch.cuda.is_available():
    device = 'cuda'
else:
    device = 'cpu'
```

- A continuación definimos la red a utilizar (MLP de dos capas). A diferencia de la definición anterior, la red no computará la softmax, ya que la misma es computada por la función de pérdida.
- Usaremos **DataLoaders** ya que si bien no son esenciales, ahorran el trabajo de dividir los datos en batches y hacer el shuffle cuando es necesario.

```
class MLP(nn.Module): #definimos una clase perceptron multi-capas
    def __init__(self, input_size, classes):
        super(MLP, self).__init__()
        self.fc1 = nn.Linear(input_size, 200) #1er capa de 200 neuronas
        self.fc2 = nn.Linear(200, classes) #2da capa de 10 neuronas (una por cada clase)

    def forward(self, x):
        out1 = self.fc1(x)
        out2 = self.fc2(out1)
        return out2

model = MLP(x_train.shape[1], 10)
model.to(device)

from torch.utils.data import TensorDataset, DataLoader

dl_train = DataLoader(TensorDataset(torch.tensor(x_train), torch.tensor(y_train, dtype=torch.long)),
                      batch_size=BS, shuffle=True)
dl_test = DataLoader(TensorDataset(torch.tensor(x_test), torch.tensor(y_test, dtype=torch.long)),
                    batch_size=BS, shuffle=False)
```

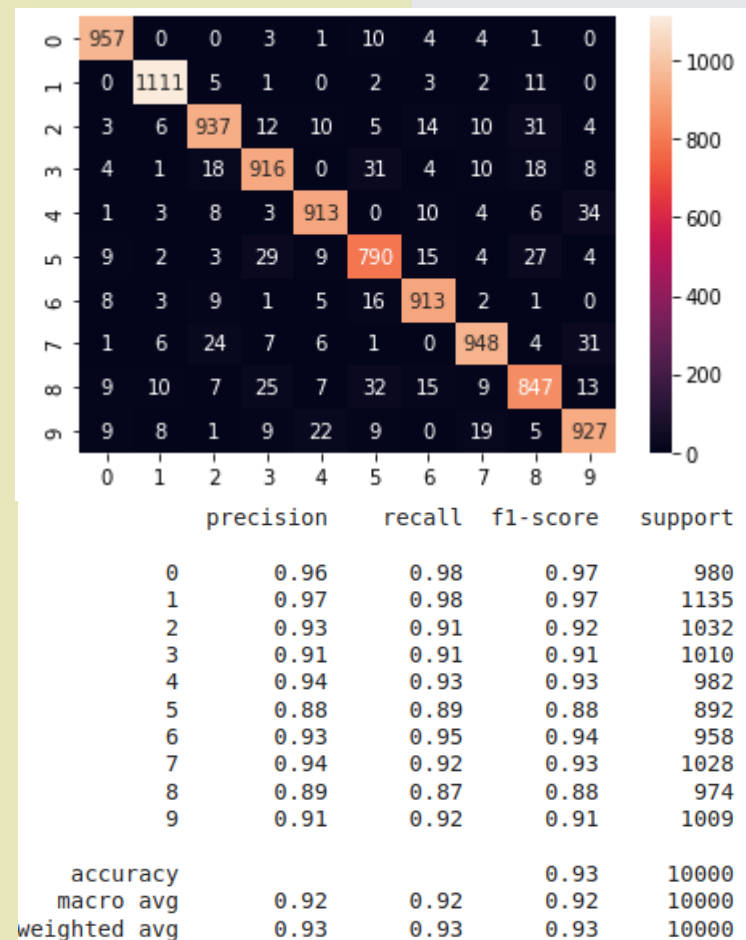
```
#defino el optimizador a usar
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9, nesterov=False)
```

```
#defino la función de pérdida
loss = nn.CrossEntropyLoss()
epochs = 100
```

```
for i in range(epochs):
    losses = []
    for x, y in dl_train:
        optimizer.zero_grad() #Resetea los gradientes
        #Mueve los datos a GPU (o CPU) y calcula la perdida
        pred = model(x.to(device))
        current_loss = loss(pred, y.to(device))
        pred = model(x) # calcula la perdida
        current_loss = loss(pred, y)
        #Calcula el gradiente
        current_loss.backward()
        optimizer.step()
        losses.append(current_loss.item())
```

```
y_pred = []
```

```
with torch.no_grad(): #No necesitamos los gradientes!
    for x, _ in tqdm(dl_test):
        pred = model(x)
        pred = nn.functional.softmax(pred, dim=1).cpu().numpy()
        y_pred.extend(np.argmax(pred, axis=-1).tolist())
```





UNIVERSIDAD
CAECE

Cámara Argentina de Comercio y Servicios